

Algorithms for Data Analysis and Data Mining

Natalia Bardadyn, Karolina Dadej, Joanna Dagil, Marcin Przesmycki

June 2, 2026

A. Data Analysis and Regression

1. Logistic Regression

(a)

The analysis of the full dataset was performed using a logistic regression model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 \cdot \text{hours}$$

The threshold for when the function reaches 0.5 and thus the binary output returns 1 was calculated using the following formula.

$$0.5 = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

$$1 + e^{-(\beta_0 + \beta_1 x)} = 2$$

$$e^{-(\beta_0 + \beta_1 x)} = 1$$

$$-(\beta_0 + \beta_1 x) = 0$$

$$x = -\beta_0 / \beta_1$$

The results show a clear relationship between study hours and the probability of success. The model provides a good fit to the data and achieves classification performance using the chosen decision threshold.

Final model function that resulted:

$$\log\left(\frac{p}{1-p}\right) = -10.83988 + 0.2091565 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability:

$$51.82663$$

(b)

The analysis of the dataset with last point removed was performed using the same model and metrics.

$$\log\left(\frac{p}{1-p}\right) = -10.83733 + 0.2091066 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability:

$$51.82679$$

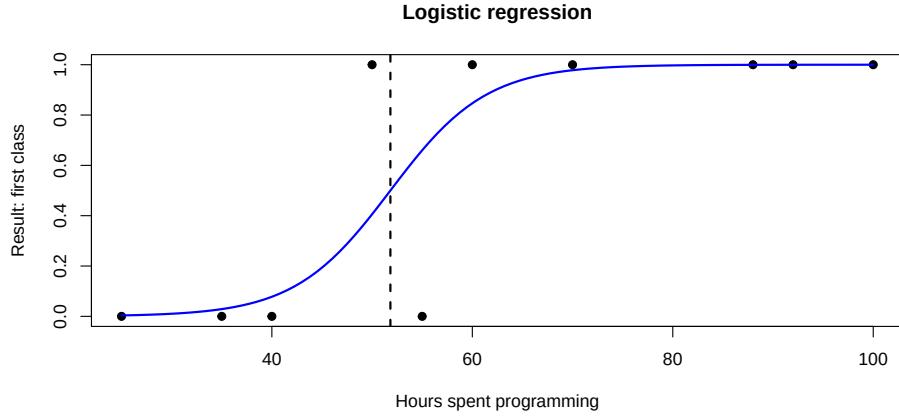


Figure 1: Logistic regression

Removing the point (100,1) does not significantly change the result. The fitted curve and the threshold remain almost the same. This is because the point (100,1) is consistent with the general pattern in the data: students with high numbers of hours usually obtained result 1.

The model already has several high-hour observations with result 1, for example 70, 88 and 92 hours. Therefore, the point (100,1) does not add much new information to the model.

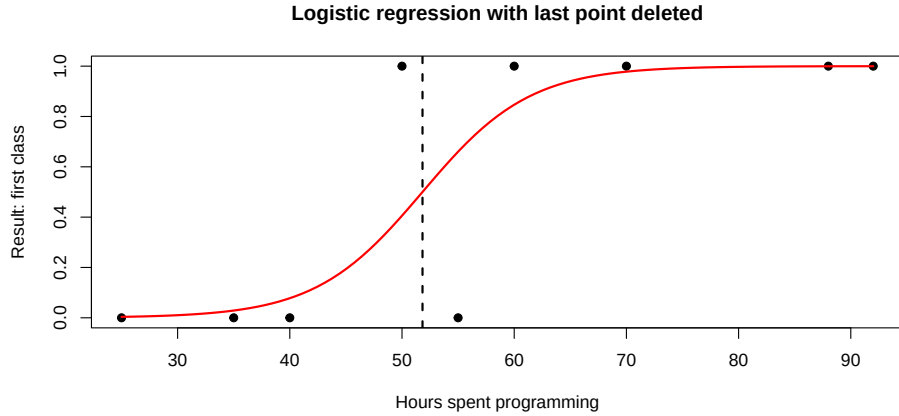


Figure 2: Logistic regression with last point deleted

(c)

The analysis of the dataset with added data point was performed using the same model and metrics.

$$\log\left(\frac{p}{1-p}\right) = -10.84002 + 0.2091594 \cdot \text{hours}$$

Final threshold for when the function reaches 0.5 probability

$$51.82662$$

Adding the point (115,1) very slightly affects the estimated model. The coefficient associated with the predictor becomes greater, indicating a steeper increase in the probability of success.

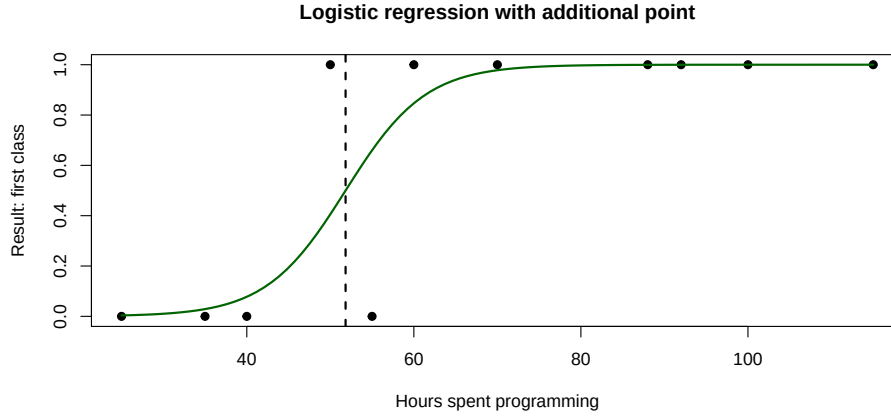


Figure 3: Logistic regression with additional point

Table 1: Comparison of logistic regression models

model	threshold	beta0	beta1
Original	51.8266	-10.8399	0.2092
Without (100,1)	51.8268	-10.8373	0.2091
With (115,1)	51.8266	-10.8400	0.2092

(d)

Logistic regression is a simple classification method for binary outcomes. One of its main advantages is that the estimated coefficients can be interpreted in terms of log-odds and odds ratios, which makes the relationship between predictors and the response variable easier to understand. Another important advantage is that the model produces predicted probabilities in the range:

$$0 \leq p \leq 1$$

making it a valuable method for probabilistic interpretation and decision-making problems. It is also computationally efficient and performs well on relatively small datasets. Lastly, it does not require the predictor variables to follow a normal distribution.

A major limitation of logistic regression is the assumption of a linear relationship between the predictors and the log-odds of the outcome:

$$\log \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x$$

If the true relationship is non-linear, the model may provide poor predictive performance. This method can also be sensitive to outliers and influential observations, especially in small datasets although the addition of the point (115,1) caused only a small change in the estimated coefficients and decision boundary. Finally, logistic regression may perform worse than more advanced machine learning methods when dealing with complex decision boundaries or large numbers of interacting variables.

2. Nonlinear Least-Squares

Three models were fitted to the data using nonlinear least squares (NLS) for f_1 and f_2 , and ordinary least squares (OLS) for the linear-in-parameters model f_3 :

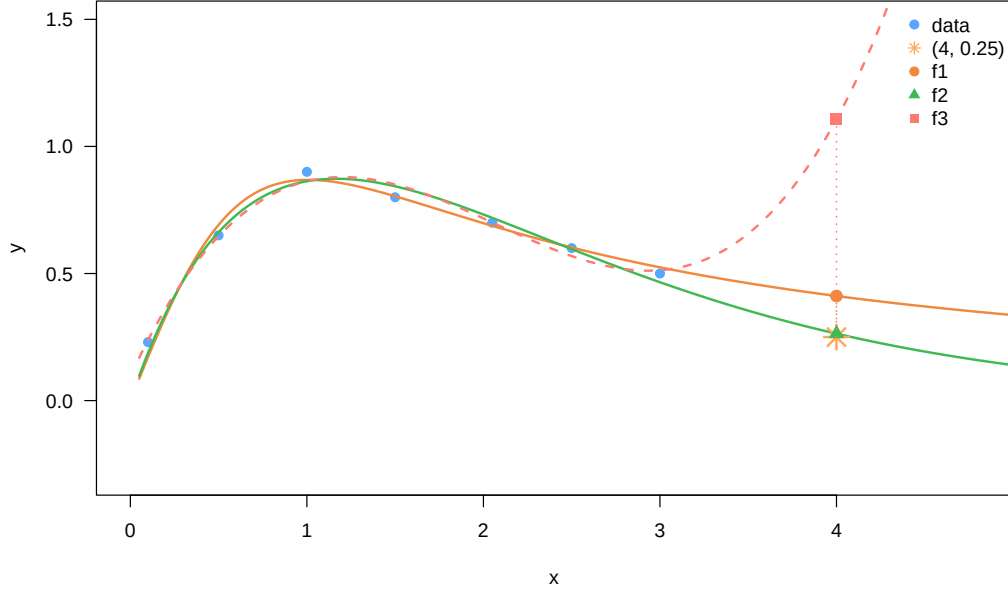
$$f_1(x) = \frac{ax}{1+cx^2}, \quad \hat{a} = 1.7233, \quad \hat{c} = 0.9845 \quad (1)$$

$$f_2(x) = Axe^{-bx}, \quad \hat{A} = 2.0350, \quad \hat{b} = 0.8577 \quad (2)$$

$$f_3(x) = a + bx + cx^2 + dx^3, \quad \hat{a} = 0.0952, \quad \hat{b} = 1.5098, \quad \hat{c} = -0.8848, \quad \hat{d} = 0.1426 \quad (3)$$

(a)

(c) Extrapolation (x=4)



All parameters are statistically significant at the 5% level. Both f_1 and f_2 converged quickly (4 and 3 iterations respectively).

Note on f_2 . The original model stated in the problem is $f_2(x) = axe^{-bx+c}$. However, this form is not identifiable: the parameters a and c are collinear since $a \cdot e^c$ constitutes a single constant. The model is therefore reparametrised as $f_2(x) = Axe^{-bx}$, where $A = ae^c$, which is mathematically equivalent and numerically well-posed.

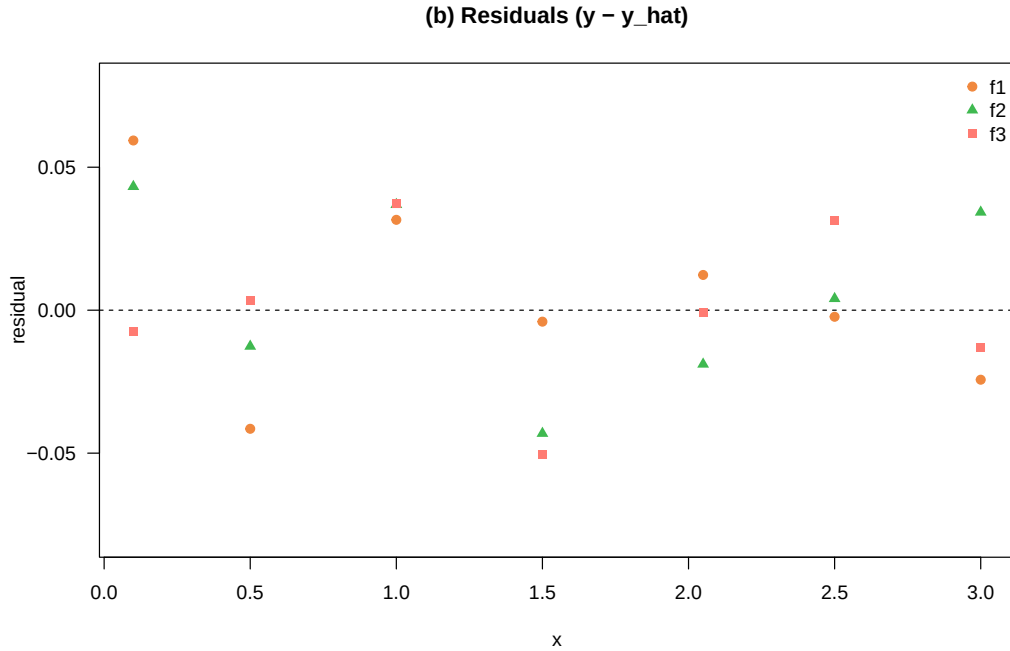
(b)

Table 2: Goodness-of-fit metrics for all three models ($n = 7$).

Model	RSS	RMSE	MAE	AIC	BIC	pseudo- R^2	#params
f_1	0.00701	0.03164	0.02505	-22.484	-22.646	0.9754	2
f_2	0.00680	0.03116	0.02758	-22.695	-22.857	0.9761	2
f_3	0.00516	0.02714	0.02052	-20.630	-20.901	0.9819	4

Although f_3 achieves the lowest RSS and highest R^2 , it uses 4 parameters compared to 2 for f_1 and f_2 . With only $n = 7$ data points, this leaves just 3 degrees of freedom, raising serious concerns about overfitting.

Comparing f_1 and f_2 directly — both with 2 parameters and 5 degrees of freedom — f_2 is superior on every criterion: lower RSS (0.00680 vs 0.00701), lower RMSE, lower AIC (-22.695 vs -22.484),



and lower BIC (-22.857 vs -22.646). The AIC and BIC explicitly penalise model complexity, so their preference for f_2 over f_1 is meaningful.

Conclusion: $f_2(x) = Axe^{-bx}$ is the best candidate. It achieves near-identical fit to f_3 with half the parameters, and outperforms f_1 on all information criteria.

(c)

Table 3: Predicted values and absolute errors at $x = 4$ (true $y = 0.25$).

Model	$\hat{y}(4)$	$ \hat{y}(4) - 0.25 $
f_1	0.4115	0.1615
f_2	0.2634	0.0134
f_3	1.1070	0.8570

f_2 predicts $\hat{y}(4) = 0.2634$, an absolute error of only 0.0134 — more than ten times smaller than f_1 and over sixty times smaller than f_3 . The catastrophic failure of f_3 ($\hat{y} = 1.107$) is a textbook illustration of polynomial extrapolation: the cubic term dominates outside the training range and the curve diverges rapidly.

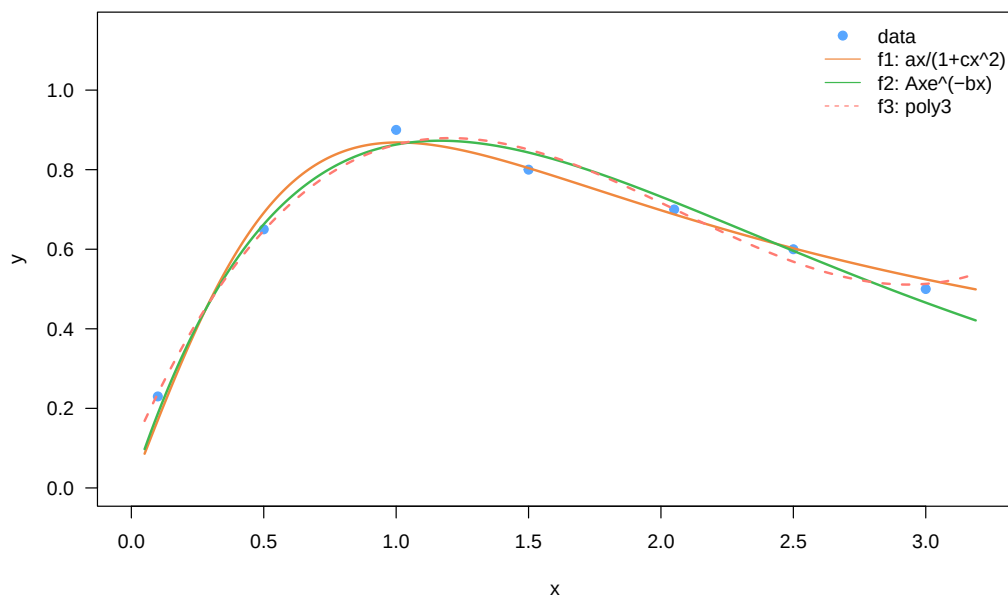
Conclusion: f_2 should be selected to minimise prediction error at $x = 4$.

(d)

Overfitting occurs when a model captures noise rather than the underlying signal, fitting the training data closely but generalising poorly to new observations. The main ways to avoid it are:

- **Information criteria (AIC, BIC):** These penalise the log-likelihood by a function of the number of parameters, rewarding parsimony. BIC applies a stronger penalty and is preferred when the true model is believed to be among the candidates.
- **Cross-validation:** Holding out part of the data to evaluate predictive performance on unseen points directly tests generalisation.

(a) Model fitting



- **Physically motivated models:** Choosing functional forms that reflect the known behaviour of the system reduces the risk of spurious parameter proliferation.
- **Residual analysis:** A well-fitted model should show structureless, homoscedastic residuals with no systematic patterns.

Assessment of the three models. With $n = 7$ observations, a rule of thumb is to allow at most $\lfloor n/5 \rfloor = 1$ to 2 free parameters for robust generalisation.

f_1 and f_2 each use 2 parameters, which is appropriate for this dataset size. f_3 uses 4 parameters, leaving only 3 residual degrees of freedom. The intercept of f_3 is not significant ($p = 0.154$), suggesting that at least one parameter is superfluous. The extrapolation result ($\hat{y}(4) = 1.107$) confirms that f_3 has more free parameters than needed and overfits the data.

B. Given Dataset

3. Data analysis

We have decided to work on the dataset `sggw_c.csv` and to perform SVM analysis.

(a) Data summary and visualization

Descriptive statistics of the data include mean, median, variance, as well as minimum and maximum.

Several variables are categorical or ordinal variables encoded as numerical values. Therefore, descriptive statistics such as mean and variance, as well as histograms and boxplots below, should be interpreted with caution.

variable	min	max	mean	median	variance
Account.Status..with.the.bank.	1	4	2.49	2.00	1.51
Credit.Months	6	60	20.07	18.00	135.10
Credit.Amount..K.1000.	3	159	31.38	21.00	746.60
Saving..account..bonds.	1	5	2.14	1.00	2.44
Years.of.Employment	1	5	3.56	3.00	1.39
Residence.Years	1	4	2.81	3.00	1.19
Age	20	74	36.43	34.00	150.76
House..1.own..2.rent.	1	4	1.44	1.00	0.36
Existing.credit.installment	1	2	1.15	1.00	0.13
Job..1.Good.	1	2	1.48	1.00	0.25
Dependent..e.g...children.	1	2	1.05	1.00	0.04
Credit..1.Good.	1	2	1.28	1.00	0.20

Table 4: Descriptive statistics of the dataset.

The data were visualized using both histograms and boxplots.

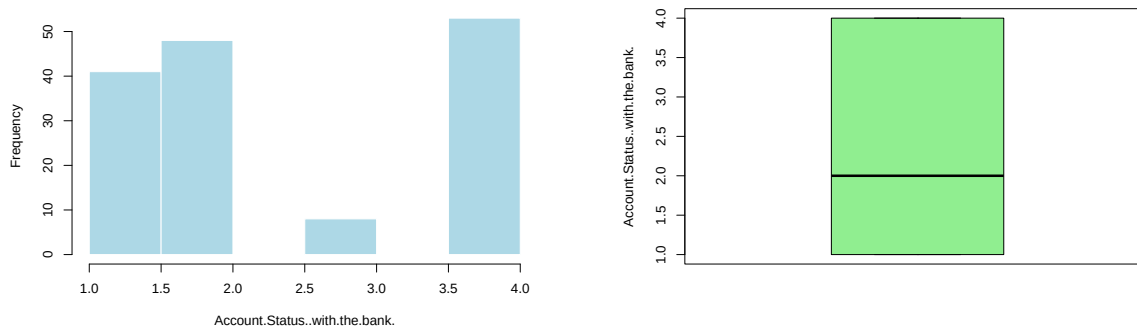


Figure 4: Distribution of Account Status with the bank

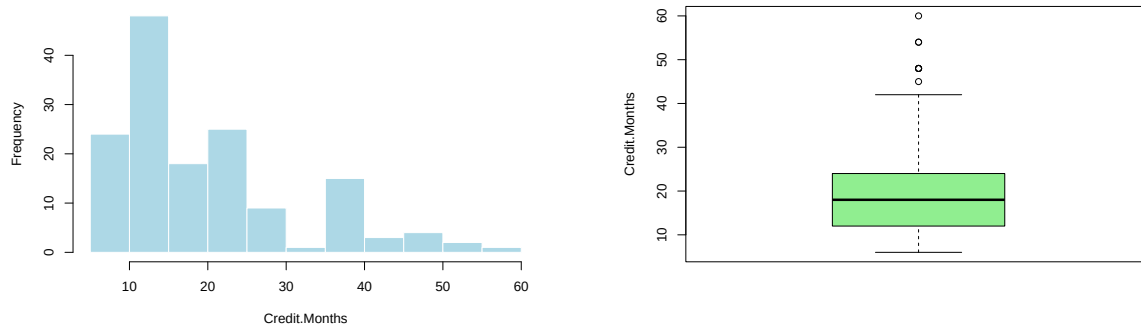


Figure 5: Distribution of Credit Months

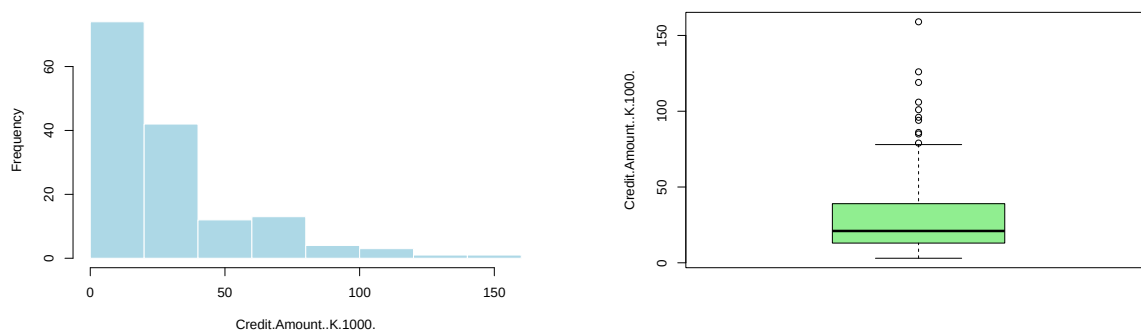


Figure 6: Distribution of Credit Amount

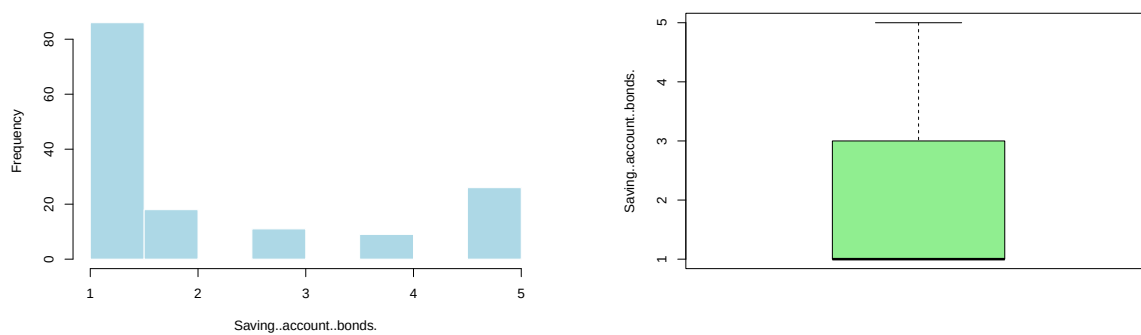


Figure 7: Distribution of Saving account bonds

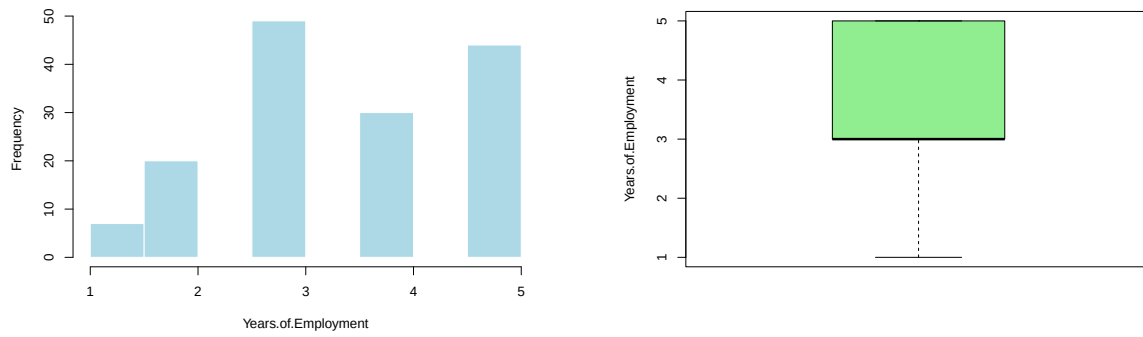


Figure 8: Distribution of Years of Employment

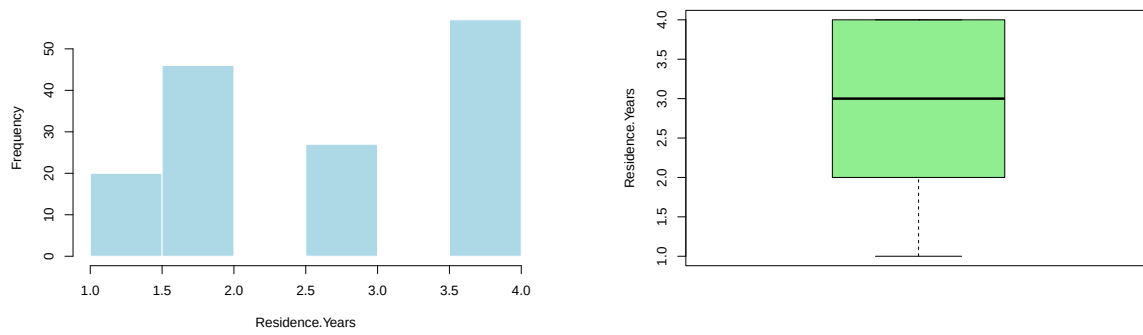


Figure 9: Distribution of Residence Years

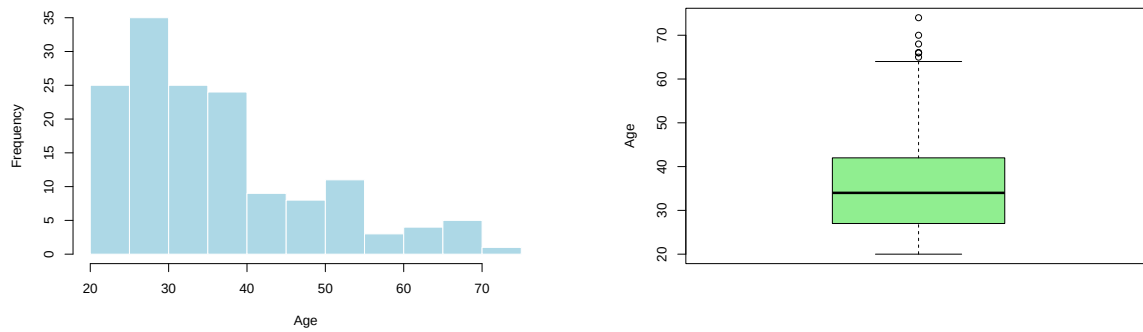


Figure 10: Distribution of Age

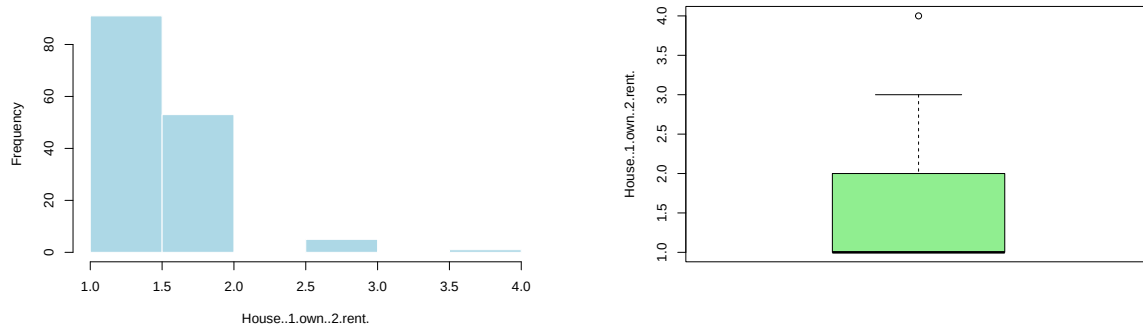


Figure 11: Distribution of House Ownership or Rental

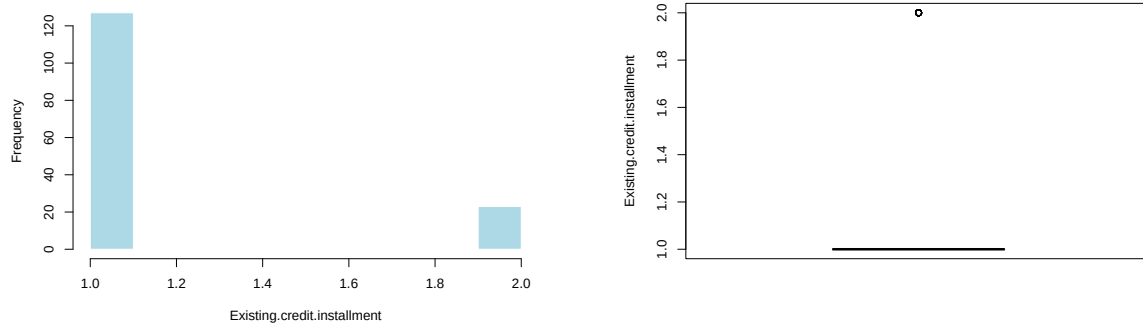


Figure 12: Distribution of Existing Credit Installments

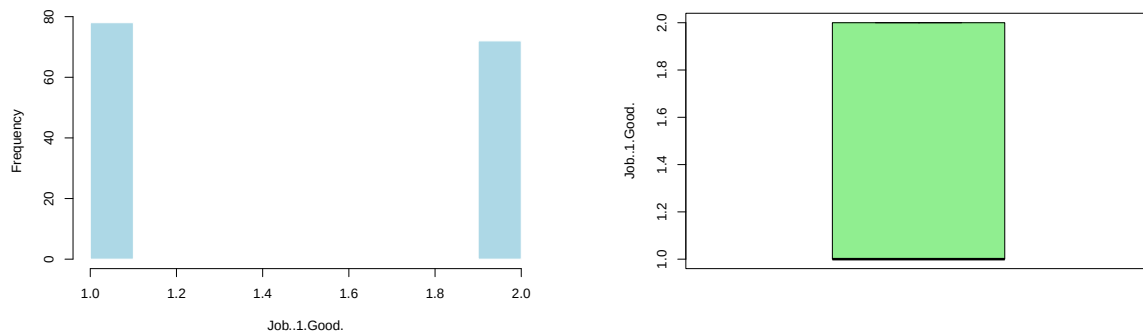


Figure 13: Distribution of Job

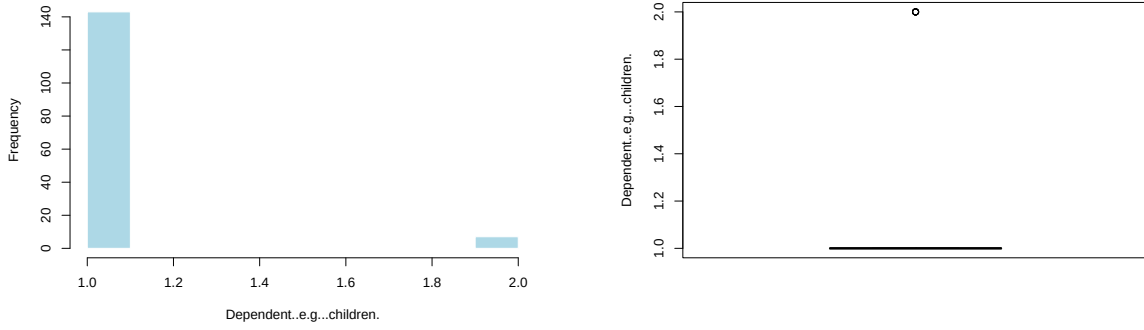


Figure 14: Distribution of Dependent

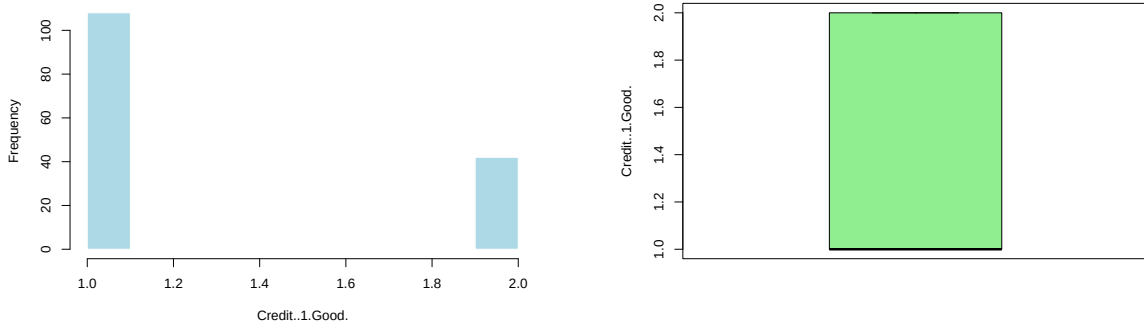


Figure 15: Distribution of Credit

(b) SVM analysis

We have chosen to perform our analysis using Support Vector Machine.

Code is heavily commented but there are a few points worth discussing further.

The dataset was split into training and test sets using an 80 : 20 proportion. Since the amount of input data was limited - 150 entries with 108 **Good** labels and 42 **Bad** labels, we decided to check the proportions of **Good** and **Bad** labels in the sampling split. With the random seed set to 42 the split was as below:

Dataset	class	count	proportion
Full dataset	Good	108	0.72
Full dataset	Bad	42	0.28
Training set	Good	87	0.72
Training set	Bad	33	0.28
Test set	Good	21	0.70
Test set	Bad	9	0.30

Table 5: Class distribution in the full dataset, training set and test set.

As it can be seen, the proportions of the original dataset are similar to the proportions in train and in test datasets, thus the split can be described as representative of the original dataset.

The standard SVM model was trained using a radial kernel. The target variable was converted to a factor before training, so the model performed classification rather than regression. The predictors were scaled, which is important for SVM because the method is sensitive to the relative scale of input variables.

(c) Results

The SVM model was trained as follows:

```
model <- svm(  
  x = data_train_x,  
  y = data_train_y,  
  kernel = "radial",  
  scale = TRUE)
```

This resulted in the following predictions:

Predicted	Good	Bad
Good	21	8
Bad	0	1

Table 6: Confusion matrix for the SVM model.

As we can see, the model works but it performs barely better than a naive model predicting only **Good** label for all cases, having accuracy at 0.7333 as compared to naive accuracy at 0.7. This is probably caused by the small dataset size, the use of default SVM parameters and class imbalance, having most of the data labeled as **Good**. Since the problem is credit risk, we could sensibly assume that the more optimal would be a model that makes less mistakes in labeling **Bad** cases as **Good** even if it has an overall lower accuracy. A **Bad** case predicted as **Good** is the dangerous error in credit risk.

We can address this by using class weights that increase the penalty for false **Good** predictions.

```
class_weights <- c(  
  Good = 1,  
  Bad = sum(data_train_y == "Good") / sum(data_train_y == "Bad")  
)  
  
model_weighted <- svm(  
  x = data_train_x,  
  y = data_train_y,  
  kernel = "radial",  
  scale = TRUE,  
  class.weights = class_weights  
)
```

The weighted model, evaluated on the same train/test split, performed as follows.

Predicted	Good	Bad
Good	16	6
Bad	5	3

Table 7: Confusion matrix for the SVM model with weighted classes.

The resulting overall accuracy is lower, but the weighted model reduces the number of **Bad** cases incorrectly classified as **Good**. Depending on the preferred types of errors the weights can be further adjusted. Below are the final metrics of the predictions of both SVM models.

	Accuracy	Precision	Bad	Recall	Bad	Specificity	F1	Bad
Standard_SVM	0.73		1.00		0.11	1.00		0.20
Weighted_SVM	0.63		0.38		0.33	0.76		0.35

Table 8: Comparison of evaluation metrics for standard and weighted SVM models.

Since the dataset is small and only one train/test split was used, the results should be interpreted as exploratory. A more reliable evaluation could use cross-validation.

C. Mini-Project

4. Classification of Morphological Features in the Dry Bean Dataset Using Support Vector Machines

1. Data Explanation and Statistical Summary

The Dry Bean Dataset consists of high-resolution camera images detailing seven distinct types of dry beans: Seker, Barbunya, Bombay, Cali, Dermosan, Horoz, and Sira. Through computer vision image processing, 16 continuous numeric features were successfully extracted from a total of 13611 instances. These attributes comprise 12 geometric/morphological dimensions and 4 specialized shape forms. The primary analytical objective is to perform multiclass classification to correctly predict the correct bean variety based entirely on these physical dimensions.

The dataset features a slight class imbalance across the categories:

- **Barbunya:** 1322 instances ($\sim 9.71\%$)
- **Bombay:** 522 instances ($\sim 3.84\%$)
- **Cali:** 1630 instances ($\sim 11.98\%$)
- **Dermason:** 3546 instances ($\sim 26.05\%$)
- **Horoz:** 1928 instances ($\sim 14.17\%$)
- **Seker:** 2027 instances ($\sim 14.89\%$)
- **Sira:** 2636 instances ($\sim 19.37\%$)

A descriptive statistical analysis of the core structural attributes outlines the physical variance present in the dataset:

- **Area (A):** Mean of 53048.28 (Std. Dev. 29324.10), spanning from 20,420.00 to 254616.00.
- **Perimeter (P):** Mean of 855.28 (Std. Dev. 214.29), spanning from 524.74 to 1985.37.
- **Major Axis Length (L):** Mean of 320.14 (Std. Dev. 85.69), spanning from 183.60 to 738.86.
- **Minor Axis Length (l):** Mean of 202.27 (Std. Dev. 44.97), spanning from 122.51 to 460.20.
- **Eccentricity (Ec):** Mean of 0.75 (Std. Dev. 0.09), spanning from 0.22 to 0.91.
- **Solidity (S):** Mean of 0.99 (Std. Dev. 0.00), spanning from 0.92 to 0.99.

The substantial standard deviations seen in features like Area and Perimeter highlight a broad structural variation in bean sizes between classes. Larger bean varieties, for example Cali and Barbunya, heavily skew the maximum limits, whereas smaller varieties, for example Dermason, populate the minimum thresholds. Furthermore, size-dependent features share massive linear correlations, specifically between Area, Perimeter, and MajorAxisLength, displaying correlation scores near 0.97–0.98. This high level of multicollinearity signals that simple linear decision models will struggle to separate highly overlapping classes.

2. Classification Methodology and Preprocessing

For this multiclass task, a Support Vector Machine (SVM) leveraging a Radial Basis Function (RBF) kernel was chosen. SVM algorithms exhibit high efficacy for this problem since the 16 extracted physical descriptors are continuous, geometric, and non-linear. The RBF kernel dynamically projects these intersecting features into a higher-dimensional hyperspace where the boundaries of overlapping varieties can be cleanly separated by an optimal hyperplane.

To maintain robust data hygiene and ensure an unbiased model evaluation, a clean preprocessing workflow was put in place:

- **Data Splitting:** The primary dataset was randomly partitioned into a Training Set (70%) to fit the model parameters, and an independent Testing Set (30%) reserved exclusively for evaluating generalizability. A static random seed (123) was used to guarantee perfect split reproducibility.
- **Feature Scaling:** Biological parameters like Area, measured in thousands of pixels, and Solidity, ranging strictly between 0 and 1, operate on entirely different scales. Because SVM hyperplanes rely directly on distance metrics, variables with larger absolute scales would dominate the model calculations. Therefore, Z-score normalization, meaning centering and scaling, was applied. Crucially, the transformation scaling vectors were extracted solely from the training data partition and subsequently projected onto both sets to fully eliminate data leakage.
- **Target Variable Encoding:** The target variable `Class` was formatted as a categorical factor to ensure the algorithm executed multiclass probabilistic classification instead of a numeric regression.

3. Hyperparameter Tuning and Critical Discussion

The default baseline SVM model was initialized utilizing a cost constraint ($C = 1$) and a kernel scaling parameter ($\gamma = 0.1$). When validated against the unseen 30% testing partition, it delivered an overall classification accuracy of approximately 92.5%–93%. Highly distinct bean varieties, such as the massive Bombay or highly spherical Seker, achieved near-perfect classification marks due to their unique shapes. Conversely, the largest concentration of prediction errors occurred between Sira and Dermason. This matches the statistical exploration in Section 1, where both classes display immense physical intersection and a size correlation close to 0.97–0.98, rendering them difficult for the hyperplane to isolate.

To optimize model generalization, a structured Grid Search was conducted. The grid evaluated 9 explicit configurations across three scales of Cost (0.1, 1, 10) and three scales of Gamma (0.01, 0.1, 1).

- **Cost (C)** manages the operational trade-off between minimizing training error and maintaining a smooth, wide decision margin. Small values provide wider margins but risk underfitting, whereas high values penalize mistakes severely, risking an overfitted boundary.
- **Gamma (γ)** defines the operational radius of influence for individual support vectors. A low value results in a wide radius where distant points are seen as similar, producing smoother boundaries, while a high value confines sample influence to localized zones, risking high-variance overfitting.

The grid search confirmed a highly non-linear relationship between the parameter choices and test set accuracy:

- **Underperforming Bounds:** At the extreme combination of $C = 0.1$ and $\gamma = 0.01$, performance degraded significantly. This occurs because a lenient error penalty paired with an overly broad radius creates a flat, overly generalized boundary that completely fails to parse overlapping classes like Sira and Dermason. Alternatively, setting a high $\gamma = 1$ caused accuracy to fall sharply as Cost climbed. This narrow radius forces complex, tight geometric pockets around single training observations, which performs poorly on testing data due to variance errors.
- **Optimal Boundary:** Peak validation performance was achieved at the configuration of Cost = 10 and $\gamma = 0.1$. At this specific intersection, the regularization penalty is strict enough to push the hyperplane to aggressively separate overlapping samples, while the kernel radius stays wide enough to construct smooth decision boundaries that generalize perfectly to unseen data.

In conclusion, an SVM classifier cannot be safely deployed on physical image data using default parameters without exploring hyperparameter spaces. For morphological dimensions displaying natural physical variations, mid-range geometric bounds work best. Forcing extreme parameters to isolate overlapping classes introduces unwanted noise. Optimal validation stability relies entirely on balancing an adequate misclassification penalty with a globally stable kernel radius.